

DBMS CASE STUDY

Quick Commerce System – Customer Management & Product Management

1. Problem Definition

Quick commerce (Q-commerce) platforms deliver essential products such as groceries, medicines, and daily-use items within a very short time frame (10–30 minutes). In such systems, **managing customer data and product data accurately and efficiently** is critical for smooth operations.

Currently, many small and medium quick commerce setups rely on:

- Manual records
- Unorganized spreadsheets
- Separate systems for customers and products

This leads to:

- Data redundancy and inconsistency
- Difficulty in tracking customer orders and purchase history
- Poor inventory control and product availability errors
- Inefficient customer service due to missing or incorrect information

Hence, there is a need for a **centralized relational database system** that can manage **customer information and product information** in an organized, secure, and scalable manner.

2. Objectives of the System

The main objectives of the proposed Quick Commerce Database System are:

1. To maintain **accurate and structured customer information** such as customer ID, name, contact details, address, and registration date.
2. To store and manage **product details** including product ID, product name, category, price, stock quantity, and availability status.
3. To establish a **relationship between customers and products** through orders for tracking purchases.
4. To reduce data redundancy using **proper normalization and relational design**.
5. To enable efficient **SQL operations in SQL*Plus**, including:
 - Data insertion and updates

- Searching and filtering records
- Aggregate queries for analysis (e.g., total products, active customers)
- 6. To support future expansion such as order management, delivery tracking, and payment modules.

3. Practical Scenario Relevance

This case study reflects a **real-world quick commerce scenario**, similar to platforms operating in urban areas where:

- Customers register on the app to place instant orders.
- Products are stored in nearby warehouses or dark stores.
- Each product has limited stock that must be updated after every order.
- Customer purchase history helps in understanding demand patterns.

Example scenario:

- A customer places an order for grocery items.
- The system checks product availability.
- Customer and product records are updated in real time.

Such operations require a **robust database system**, making this case study highly relevant for practical DBMS implementation.

4. System Scope Identified

Included in Scope

- Customer Management
 - Customer registration
 - Storage of customer details
 - Viewing and updating customer information
- Product Management
 - Product catalog maintenance
 - Stock quantity tracking
 - Product price and category management
- Database Operations using SQL*Plus
 - Table creation with primary and foreign keys
 - Data insertion (minimum 5 records per table)
 - Execution of SQL queries for retrieval and analysis